

DATABASE SYSTEMS - PHASE 2 PROJECT REPORT

Space Booking System

Group 03

Member	Student ID
Trinh Van Thanh	24125044
Le Duc Thinh	24125045
Nguyen Hoang Gia	24125099
Tran Duc Thien	24125081

Course: CS486 - Introduction to Database Systems
Instructor: Dr. Le Thi Nhan
Semester: Semester 6
Submission Date: 10th August 2026

Contents

1	Introduction	3
1.1	Phase 1 Background	3
1.2	Phase 2 Motivation	3
2	Group Members and Contributions	3
3	Updated Database Design	4
3.1	Updated ERD	4
3.2	Schema Changes	7
3.3	Important Design Decisions	7
4	Concurrency Conflict Analysis	8
4.1	Business Invariant	8
4.2	Unsafe Check-Then-Write Schedule	8
4.3	Conflicting Operation Paths	9
5	Concurrency Solution	9
5.1	Selected Locking Protocol	9
5.2	Implemented Transaction Paths	10
6	Concurrency Test Results	11
6.1	Test Environment and Method	11
6.2	Recorded Results	11
6.3	Verification and Conclusion	11
7	Analytical Queries	12
7.1	Q1: Total Approved Booking Hours per Space	12
7.2	Q2: Approved Bookings by Weekday and Hour	13
7.3	Q3: Available Space Finder	13
7.4	Q4: Bookings Affected by Maintenance Escalation	14
7.5	Analytical Query Validation	14
8	Indexing and Query Tuning	14
8.1	Queries Selected for Tuning	14

8.2	Baseline	14
8.3	Index Design	15
8.4	After Indexing	16
9	Functional Dependencies	16
9.1	How to Read the FD Tables	16
9.2	Functional Dependencies by Relation	16
9.3	FD-Based 3NF Observation	20
10	Normalization Validation	20
10.1	Validation Criteria	20
10.2	Relation-Level Validation	21
10.3	Important Design Checks	21
10.4	Final Result	22
11	Agent and LLM Improvement Process	22
11.1	LLM Models	22
11.2	Improvements from Phase 1	22

1 Introduction

1.1 Phase 1 Background

Phase 1 established the baseline database for a campus space booking system. The main stored facts included user accounts, spaces, booking requests, approval decisions, usage sessions, maintenance records, and facilities. The baseline schema supported the ordinary booking lifecycle: users submit booking requests, staff review requests, approved bookings may lead to usage sessions, and maintenance records describe spaces that should not be booked while unavailable.

This report does not repeat the full Phase 1 analysis. Phase 2 is evaluated as an extension of that baseline, using the Phase 1 schema and artifacts as the implementation foundation.

1.2 Phase 2 Motivation

The Phase 2 requirement changes three major areas of the system. First, maintenance is no longer a single blocking condition. Out-of-service maintenance still blocks overlapping bookings, while advisory maintenance allows booking but requires requester notification and acknowledgement. Second, selected booking requests may be approved instantly, while other requests still follow staff approval. Third, the Facility Manager requires four analytical reports based on accumulated booking and maintenance history.

The most important correctness concern is concurrency. The requirement states that two approved bookings cannot use the same space during overlapping time periods, regardless of whether the bookings are created through instant approval or staff approval. Therefore the database design, migration, stored procedures, and tests must treat availability checking and approval writing as one protected operation.

2 Group Members and Contributions

Table 1: Group members and individual analytical query assignments

Member	Student ID	Query	Description
Trinh Van Thanh	24125044	Q1	Total approved booking hours per space for a given semester.
Le Duc Thinh	24125045	Q2	Number of approved bookings by weekday and hour for a given semester.
Nguyen Hoang Gia	24125099	Q3	Available-space finder by capacity, facility list, and requested time period.
Tran Duc Thien	24125081	Q4	Approved bookings affected when a maintenance record is escalated to out-of-service.

3 Updated Database Design

3.1 Updated ERD

The updated ERD preserves the Phase 1 data model and introduces only the additional persistent facts required for maintenance impact levels, maintenance-impact history, advisory acknowledgements, and instant-approval configuration. The canonical Mermaid `erDiagram` is maintained in artifact 09. (Figure 1)

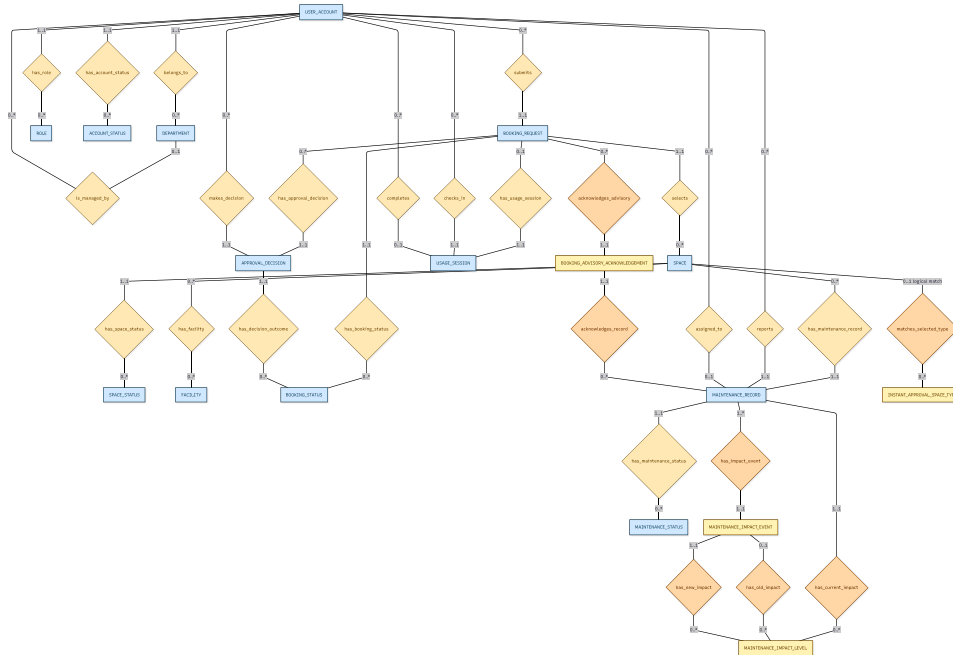


Figure 1: Updated Phase 2 ERD

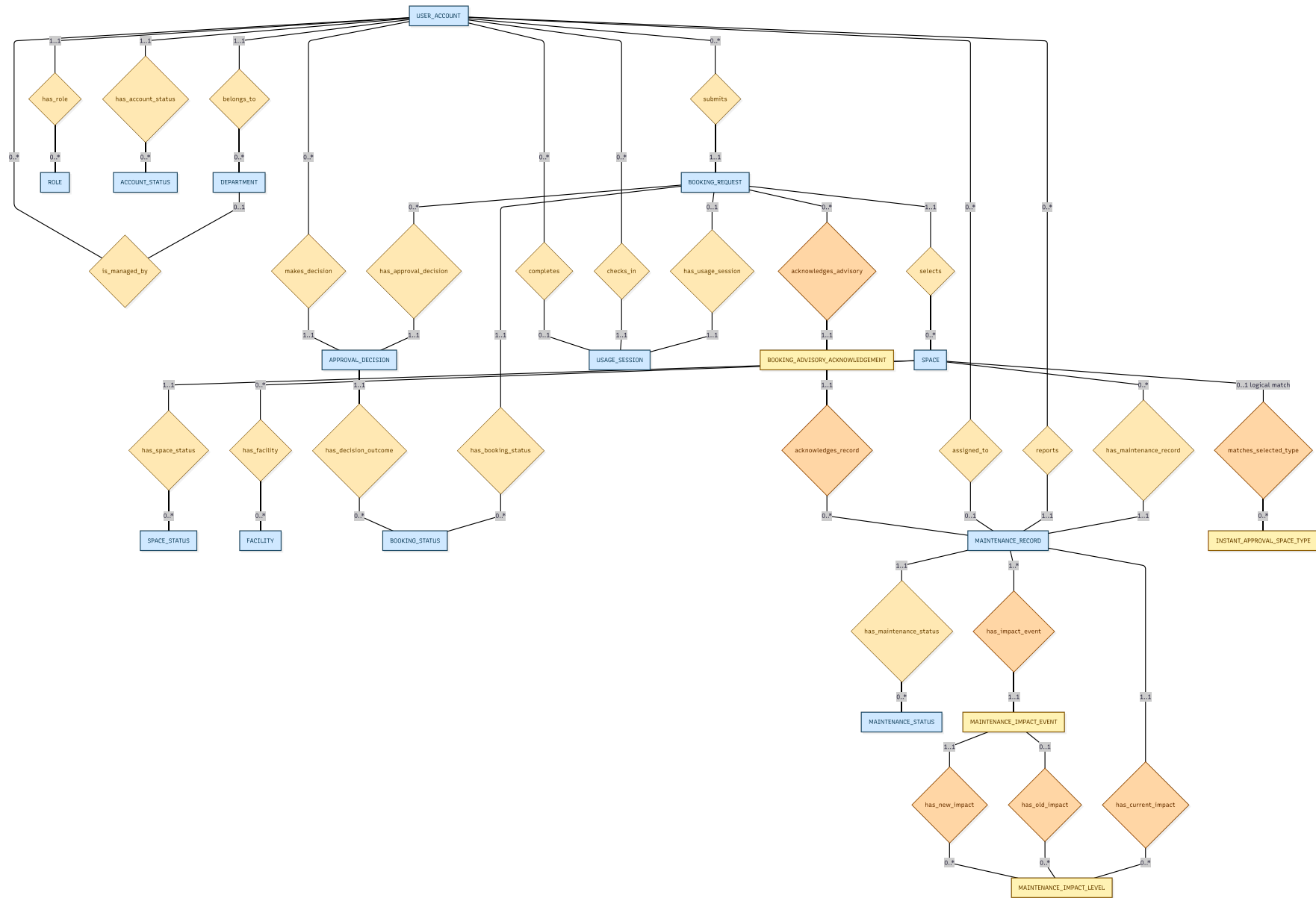


Figure 2: Updated Phase 2 ERD

3.2 Schema Changes

The Phase 2 design keeps most Phase 1 relations unchanged and introduces only the tables and columns needed to satisfy the new requirements (Table 2)

Table 2: Phase 2 schema change inventory

Change Type	Relations
Modified	BOOKING_STATUS, MAINTENANCE_RECORD
Expanded use, structure unchanged	ROLE, USER_ACCOUNT, APPROVAL_DECISION
Added	MAINTENANCE_IMPACT_LEVEL, MAINTENANCE_IMPACT_EVENT, BOOKING_ADVISORY_ACKNOWLEDGEMENT, INSTANT_APPROVAL_SPACE_TYPE

3.3 Important Design Decisions

Advisory acknowledgement is modeled as a separate relation instead of a single `booking.advisory_acknowledged` flag. A space may have several active advisory maintenance records at the same time, and each booking must record which active overlapping advisory records were acknowledged at booking time. The relation `BOOKING_ADVISORY_ACKNOWLEDGEMENT` links a booking request to the relevant maintenance record.

Maintenance impact is represented in two layers. `MAINTENANCE_RECORD` stores the current impact level so current availability checks can be evaluated directly. `MAINTENANCE_IMPACT_EVENT` stores the history of impact-level changes so the system can identify approved bookings affected by escalation to out-of-service.

Automatic and staff approvals reuse the existing `APPROVAL_DECISION` relation rather than introducing a new approval-method table. Staff decisions reference the responsible staff account, while an automatically approved booking references a dedicated active user whose role is System. This preserves the Phase 1 approval-history structure while allowing automatic decisions to remain auditable.

No `ACADEMIC_SEMESTER` relation is introduced. Phase 2 requires semester-scoped reports but does not require semester data to be persisted. Therefore, analytical queries receive explicit semester start and end bounds as parameters. This keeps the Phase 2 migration additive and avoids introducing a master entity that is not required by the source.

Instant-approval scope is represented by `INSTANT_APPROVAL_SPACE_TYPE`. Each row selects one existing `SPACE.space_type` value for the instant path. No separate `SPACE_TYPE` master relation is introduced because Phase 1 stores `space_type` as an open text attribute, and converting it into a new master entity would make the Phase 2 migration unnecessarily invasive. The exact selected values remain deployment configuration.

Concurrency mechanisms are intentionally excluded from the ERD. The requirement that two approved bookings must never overlap is a cross-row, cross-transaction invariant and

cannot be represented as an ordinary row-level constraint. Locking, transaction ordering, retry behavior, and maintenance/booking atomicity are therefore implemented in the concurrency artifacts rather than modeled as additional database entities.

Brief description of each design element is shown in Table 3

Table 3: Phase 2 Purpose of design elements

Design element	Purpose
<code>BOOKING_STATUS.status_code</code>	Stable machine-readable status predicates
<code>MAINTENANCE_RECORD.impact_level_id</code>	Current advisory/out-of-service state
<code>MAINTENANCE_IMPACT_LEVEL</code>	Controlled impact domain
<code>MAINTENANCE_IMPACT_EVENT</code>	Impact-change history and escalation reporting
<code>BOOKING_ADVISORY_ACKNOWLEDGEMENT</code>	Booking-specific advisory acknowledgement
<code>INSTANT_APPROVAL_SPACE_TYPE</code>	Configures space types eligible for instant path
System role/user	Audit actor for automatic approvals

4 Concurrency Conflict Analysis

4.1 Business Invariant

Two committed bookings in `approved` or `checked_in` status must not use the same space during overlapping periods.

Booking intervals use the half-open convention [proposed—not stated in source]. For different bookings (A) and (B) in the same space, an overlap exists when

$$A_{start} < B_{end} \wedge A_{end} > B_{start}.$$

Therefore, one booking may begin exactly when another ends. Pending, rejected, cancelled, completed, and no-show bookings do not occupy the current schedule under the approved demo rule.

4.2 Unsafe Check-Then-Write Schedule

Under an unprotected `READ COMMITTED` workflow, two transactions can both observe an empty conflict range before either approval becomes visible.

Table 4: Unsafe same-space approval schedule

Time	Transaction T1	Transaction T2
t1	Checks the requested interval; no conflict is visible.	
t2		Checks an overlapping interval; no conflict is visible.
t3	Records the first booking as approved.	
t4		Records the second booking as approved.
t5	Commits.	Commits.

The final state violates the invariant even though each transaction passed its own availability check. Locking only existing booking rows is insufficient because the conflicting range may initially contain no rows to lock.

4.3 Conflicting Operation Paths

Table 5: Concurrency paths that share the same protected space domain

Path combination	Race	Required outcome
Instant / instant	Two submissions independently choose instant approval.	At most one approval may commit.
Instant / staff	Submission and staff decision approve overlapping requests.	At most one approval may commit.
Staff / staff	Two staff members approve different pending requests.	At most one approval may commit.

All approval paths must therefore share one protocol. Maintenance changes use the same space protection so an approval cannot observe inconsistent impact state and the current impact, event, and affected-booking result stay atomic. Demo instant eligibility uses configured types and capacity; the exact types remain an open policy question.

5 Concurrency Solution

5.1 Selected Locking Protocol

The implementation uses a transaction-owned exclusive SQL Server application lock for each space. The resource name is `G03:approved-occupancy:space:` followed by the numeric `space_id`, with database principal `public`. Operations for the same space serialize, while operations for different spaces use different resources and may proceed concurrently.

Every protected operation follows the same retained-lock order:

1. Discover `space_id` without retaining a row lock when starting from a booking or maintenance identifier.
2. Begin the transaction and acquire the exclusive, transaction-owned `sp_getapplock` resource for that space.
3. Lock and recheck `SPACE` using `UPDLOCK` and `HOLDLOCK`, then lock and recheck the dependent row and its space.
4. Revalidate all business rules, write atomically, and either commit or roll back and rethrow.

This order prevents booking-first and maintenance-first lock inversions. It also protects an empty conflict range, which ordinary row locks cannot do.

5.2 Implemented Transaction Paths

Table 6: Concurrency procedures and their atomic responsibilities

Procedure	Protected behavior
<code>usp_G03_SubmitBooking</code>	Rechecks space availability, active out-of-service maintenance, instant eligibility, occupancy, and advisories; inserts the booking, acknowledgements, and any System decision atomically.
<code>usp_G03_DecideBooking</code>	Allows active facility staff or managers to decide only a pending request. Approval rechecks space, occupancy, and out-of-service maintenance; status and decision history commit together.
<code>usp_G03_ChangeMaintenanceImpact</code>	Changes only active maintenance between <code>advisory</code> and <code>out_of_service</code> ; updates current impact and its event at one timestamp; escalation returns affected occupancy without cancelling bookings.

All three procedures use half-open overlap checks and treat `approved` and `checked_in` as current occupancy. They resolve meanings through stable text rather than guessed identity values. Generated timestamps use Vietnam-local wall-clock time through `SE Asia Standard Time`.

After T1 acquires the resource, T2 waits, then rechecks the committed state. If T1 approved an overlap, T2 returns 52103. All supported paths use this namespace and acquire their own locks; direct table writes remain outside the protected interface. Production contains no `WAITFOR`, retry framework, permissions framework, or performance indexes because these are outside Artifact 12's scope. Exact instant-approved space types and authorization for maintenance-impact changes remain open policy questions.

6 Concurrency Test Results

6.1 Test Environment and Method

The required two-session suite was executed on 10 August 2026 using SQL Server 2022 Developer Edition 16.0.4255.1 (CU25) on Linux, database compatibility level 160. Testing used a disposable database named `G03LocalTimeGeneratorVerify20260810`, which was removed after evidence capture. Artifacts 05, 06, 10, and 12 were deployed before the test setup.

Setup created deterministic `G03-CT-*` users, spaces, and bookings. The unsafe scripts deliberately performed a naive check-then-write race. For each protected case, Window A started first and Window B started within five seconds. A test-only trigger delayed A after the production procedure reached its booking write and had acquired its own lock. The trigger never acquired an application lock, and both windows called the production procedures directly.

6.2 Recorded Results

Table 7: Actual committed-state concurrency evidence

Executed case	Window results	Verifier result	Status
Unsafe instant / instant	Both naive sessions committed.	One overlapping approved pair found.	PASS
Safe instant / instant	A committed; B returned 52103.	Zero overlap pairs.	PASS
Safe instant / staff	Instant A committed; staff B returned 52103.	Zero overlap pairs.	PASS
Safe staff / staff	Staff A committed; staff B returned 52103.	Zero overlap pairs.	PASS

The unsafe result demonstrates that an availability check under an unprotected workflow does not preserve the invariant. In every safe case, B waited for A’s procedure-owned space lock, acquired the resource after A committed, rechecked the committed overlap, and returned the required conflict error 52103. Error 52104 was not accepted as equivalent evidence because it indicates an invalid fixture or lifecycle state rather than an overlap detected after serialization.

6.3 Verification and Conclusion

Each case used one classroom repetition. After both sessions finished, the verifier inspected committed `approved` and `checked_in` bookings using the half-open overlap predicate. It found one overlap for the deliberately unsafe fixture and zero overlaps across all protected fixtures. The production procedures contained no `WAITFOR`; deterministic

delay was confined to the test-only trigger. The cleanup script is scoped to dropping that trigger and removing only `G03-CT-*` fixtures.

The executed evidence therefore records a final result of `PASS`: the race was reproduced before protection, and the shared production protocol preserved the no-overlap invariant for instant/instant, instant/staff, and staff/staff approval combinations.

7 Analytical Queries

Artifact 16 implements the four Phase 2 reports as SQL Server stored procedures. The common query semantics are consistent with artifact 09: intervals are half-open, business timestamps are already stored as Vietnam-local `DATETIME2` values, semester input is supplied as scalar start and end bounds, and historical approval is identified by an approved row in `APPROVAL_DECISION`. Current room occupancy for availability checks is limited to bookings whose current status code is `approved` or `checked_in`.

Table 8: Implemented analytical procedures

Report	Stored Procedure
Q1: Approved hours by space	<code>dbo.usp_G03_ReportApprovedHoursBySpace</code>
Q2: Approved booking starts by weekday/hour	<code>dbo.usp_G03_ReportApprovedBookingStartsByWeekdayHour</code>
Q3: Available-space finder	<code>dbo.usp_G03_FindAvailableSpaces</code>
Q4: Bookings affected by escalation	<code>dbo.usp_G03_ReportBookingsAffectedByEscalation</code>

7.1 Q1: Total Approved Booking Hours per Space

Procedure: `dbo.usp_G03_ReportApprovedHoursBySpace`

Purpose: return total approved booking hours for every space within a supplied semester interval. The procedure accepts `@semester_start` and `@semester_end`; there is no persisted `ACADEMIC_SEMESTER` table in the Phase 2 design.

The procedure returns `space_id`, `unique_space_code`, `space_name`, and `approved_hours`. It includes every space, even if the approved total is zero. Bookings that cross the semester boundary are clipped to the requested bounds before the duration is converted to decimal hours.

Key tables: `SPACE`, `BOOKING_REQUEST`, `APPROVAL_DECISION`, and `BOOKING_STATUS`.

Important rule: the report uses `EXISTS` over `APPROVAL_DECISION` where the decision outcome status code is `approved`. This means later lifecycle states such as `completed`, `cancelled`, or `no_show` do not erase the fact that the booking was historically approved.

7.2 Q2: Approved Bookings by Weekday and Hour

Procedure: `dbo.usp_G03_ReportApprovedBookingStartsByWeekdayHour`

Purpose: count approved booking starts by weekday and hour for a supplied semester interval. The procedure accepts `@semester_start` and `@semester_end` and validates that the interval is non-empty.

The output columns are `weekday_number`, `weekday_name`, `start_hour`, and `approved_booking_count`. Each booking is counted once in the bucket of its requested start time. Weekday numbering is deterministic: Monday is 1 and Sunday is 7, independent of SQL Server `DATEFIRST` or language settings.

Key tables: `BOOKING_REQUEST`, `APPROVAL_DECISION`, and `BOOKING_STATUS`.

Important rule: as in Q1, approval is determined from approved decision history rather than from only the current booking status.

7.3 Q3: Available Space Finder

Procedure: `dbo.usp_G03_FindAvailableSpaces`

Purpose: find spaces that satisfy required capacity, required facilities, and a requested time interval.

Inputs are `@requested_start`, `@requested_end`, `@required_capacity`, and `@required_facility_ids_json`. Facility input is a JSON array of integer facility IDs. The implementation uses `OPENJSON` to parse this input relationally; an empty JSON array means no facility restriction. Non-integer array values are rejected.

The output includes `space_id`, `unique_space_code`, `space_name`, `capacity`, `space_type`, `usage_policy`, and `active_advisory_count`.

Key tables: `SPACE`, `SPACE_STATUS`, `SPACE_FACILITY`, `BOOKING_REQUEST`, `BOOKING_STATUS`, `MAINTENANCE_RECORD`, `MAINTENANCE_STATUS`, and `MAINTENANCE_IMPACT_LEVEL`.

Availability conditions:

- the space capacity must be at least the requested capacity;
- the current space status must not be `temporarily closed` or `retired`;
- every requested facility ID must exist for the space in `SPACE_FACILITY`;
- no current `approved` or `checked_in` booking may overlap the requested interval;
- no active `out_of_service` maintenance record may overlap the requested interval;
- active `advisory` maintenance does not exclude the space, but the count is returned so a later booking path can notify and acknowledge the advisory.

7.4 Q4: Bookings Affected by Maintenance Escalation

Procedure: `dbo.usp_G03_ReportBookingsAffectedByEscalation`

Purpose: identify bookings affected when a maintenance impact event changes a record from `advisory` to `out_of_service`.

The procedure accepts `@maintenance_impact_event_id`. It requires an actual event whose old impact is `advisory` and whose new impact is `out_of_service`. Other events are rejected.

The output includes booking and requester contact details: `booking_request_id`, `requester_user_account_id`, `user_id`, `full_name`, `email`, `phone_number`, requested start/end time, current status code, and escalation timestamp.

Key tables: `MAINTENANCE_IMPACT_EVENT`, `MAINTENANCE_RECORD`, `MAINTENANCE_IMPACT_LEVEL`, `BOOKING_REQUEST`, `BOOKING_STATUS`, `APPROVAL_DECISION`, and `USER_ACCOUNT`.

Important rule: affected bookings must have an approved decision at or before the escalation time and must overlap the affected interval beginning at `max(changed_at, maintenance.start_time)`. The report supports the Phase 2 requirement that staff can identify requesters to contact; it does not automatically cancel or modify those bookings.

7.5 Analytical Query Validation

Artifact 14 records a post-regeneration analytical smoke test using the generated `G03-GEN-V2` dataset. The smoke test passed: Q1 returned 109 spaces, Q2 returned 28 weekday/hour buckets, a representative room finder returned 100 spaces, and a generated escalation event returned 32 affected bookings.

8 Indexing and Query Tuning

8.1 Queries Selected for Tuning

The Phase 2 requirement requires tuning the booking conflict check, the room finder, and two reporting queries other than the room finder. The measured workload consists of:

1. Booking conflict check for approved-booking overlap.
2. Q3: Available-space finder.
3. Q1: Total approved booking hours per space.
4. Q2: Approved bookings by weekday and hour.

8.2 Baseline

The benchmark was executed on SQL Server 2022 Developer Edition 16.0.4255.1 at compatibility level 160. A clean database was rebuilt from the current artifacts and validated

with 100,000 generated bookings before measurement. One warm-up pass preceded the measured pass. Actual-row profiles were captured separately so that plan serialization did not affect the reported times. Runtime statistics were enabled with:

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

Table 9: Baseline metrics before indexing

Query	Elapsed Time	CPU Time	Logical Reads	Main Operator
Conflict check	7 ms	7 ms	BR: 833	Clustered index scan
Room finder	430 ms	430 ms	BR: 83,300; MR: 800; SF: 208	Clustered index scans
Report Q1	42 ms	43 ms	BR: 833; AD: 864	Clustered index scans
Report Q2	34 ms	34 ms	BR: 833; AD: 864	Clustered index scans

Here BR denotes `BOOKING_REQUEST`, MR denotes `MAINTENANCE_RECORD`, SF denotes `SPACE_FACILITY`, and AD denotes `APPROVAL_DECISION`.

8.3 Index Design

Four nonclustered B-tree indexes were retained from the measured workload:

- `IX_BOOKING_REQUEST_space_status_start` on `(space_id, booking_status_id, requested_start_time)` with `requested_end_time` included. It supports the conflict check and the booking-overlap part of the room finder.
- `IX_MAINTENANCE_RECORD_space_status_impact_start` on `(space_id, maintenance_status_id, impact_level_id, start_time)` with `completion_time` included. It supports advisory and out-of-service overlap checks.
- `IX_BOOKING_REQUEST_start_space` on `(requested_start_time, space_id)` with `requested_end_time` included. It supports semester range access for Q1 and Q2.
- `IX_APPROVAL_DECISION_outcome_booking_time` on `(decision_outcome_booking_status_id, booking_request_id, decision_time)`. It covers historical approval lookup for Q1 and Q2.

The existing unique nonclustered index on `SPACE_FACILITY(space_id, facility_id)` already supports facility matching, so no redundant facility index was added.

8.4 After Indexing

The same ordered result output was captured before and after indexing. Both files contained 250 lines and had identical SHA-256 values; the full digest is retained in Artifact 15. The improvement column below distinguishes elapsed-time reduction from logical-read reduction. A displayed time of 0 ms means below SQL Server’s millisecond timer resolution, not zero work.

Table 10: Before-and-after tuning comparison

Query	Before	After	Improvement
Conflict check	7 ms; BR 833 reads	0 ms displayed; BR 6 reads	BR reads reduced 99.3%; elapsed below 1 ms
Room finder	430 ms; BR 83,300; MR 800	3 ms; BR 700; MR 400	Elapsed reduced 99.3%; BR 99.2%; MR 50.0%
Report Q1	42 ms; BR 833; AD 864	40 ms; BR 111; AD 181	Elapsed reduced 4.8%; BR 86.7%; AD 79.1%
Report Q2	34 ms; BR 833; AD 864	28 ms; BR 111; AD 181	Elapsed reduced 17.6%; BR 86.7%; AD 79.1%

9 Functional Dependencies

Artifact 09 identifies the functional dependencies (FDs) used to validate the final schema. An FD $X \rightarrow Y$ means that one value of X determines exactly one value of Y . Only FDs supported by the declared primary-key and unique constraints in Artifacts 05 and 10 are used below.

9.1 How to Read the FD Tables

Each row explicitly shows a determinant and the non-trivial attributes it determines. Trivial FDs and FDs obtained only by augmentation are omitted. If a relation has several candidate keys, one row is shown for each key.

3NF reading rule. For every listed FD $X \rightarrow A$, the determinant X is a candidate key. Therefore no non-key attribute determines another non-key attribute, and no transitive dependency appears among the documented FDs.

9.2 Functional Dependencies by Relation

Lookup and Configuration Relations

Table 11: Explicit FDs for lookup and configuration relations

Relation	Determinant	Determined attributes
ROLE	role_id	role_name
	role_name	role_id
ACCOUNT_STATUS	account_status_id	status_name
	status_name	account_status_id
SPACE_STATUS	space_status_id	status_name
	status_name	space_status_id
BOOKING_STATUS	booking_status_id	status_code, status_name
	status_code	booking_status_id, status_name
	status_name	booking_status_id, status_code
MAINTENANCE_STATUS	maintenance_status_id	status_name
	status_name	maintenance_status_id
MAINTENANCE_IMPACT_LEVEL	impact_level_id	impact_level_code
	impact_level_code	impact_level_id
INSTANT_APPROVAL_SPACE_TYPE	instant_approval_space_type_id	space_type
	space_type	instant_approval_space_type_id

All determinants in this group are candidate keys. In particular, `BOOKING_STATUS.status_code` becomes an alternate key through the unique constraint added by Artifact 10.

Core Entity Relations

Table 12: Explicit FDs for core entity relations

Relation	Determinant	Determined attributes
DEPARTMENT	department_id	department_name, head_user_account_id
	department_name	department_id, head_user_account_id
USER_ACCOUNT	user_account_id	user_id, full_name, email, phone_number, department_id, role_id, account_status_id

Relation	Determinant	Determined attributes
SPACE	user_id	user_account_id, full_name, email, phone_number, department_id, role_id, account_status_id
	email	user_account_id, user_id, full_name, phone_number, department_id, role_id, account_status_id
	space_id	unique_space_code, space_name, space_type, building, floor, room_number, capacity, usage_policy, space_status_id
	unique_space_code	space_id, space_name, space_type, building, floor, room_number, capacity, usage_policy, space_status_id
FACILITY	facility_id	facility_name

No FD is asserted from SPACE.space_type to capacity, policy, or any other space attribute. Likewise, FACILITY.facility_name is not a determinant because the implemented schema does not declare it unique.

Booking and Usage Relations

Table 13: Explicit FDs for booking and usage relations

Relation	Determinant	Determined attributes
BOOKING_REQUEST	booking_request_id	requester_user_account_id, space_id, booking_status_id, requested_start_time, requested_end_time, purpose_of_use, expected_number_of_participants
APPROVAL_DECISION	approval_decision_id	booking_request_id, decided_by_user_account_id, decision_outcome, booking_status_id, decision_time, decision_note, rejection_reason
USAGE_SESSION	usage_session_id	booking_request_id, checked_in_by_user_account_id, completed_by_user_account_id, actual_start_time, initial_condition_of_space, actual_end_time, final_condition_of_space, usage_notes

Relation	Determinant	Determined attributes
	booking_request_id	usage_session_id, checked_in_by_user_account_id, completed_by_user_account_id, actual_start_time, initial_condition_of_space, actual_end_time, final_condition_of_space, usage_notes

APPROVAL_DECISION.booking_request_id is deliberately not unique, so it is not a determinant. This permits a booking to retain multiple decision history rows without introducing an invalid FD.

Maintenance Relations

Table 14: Explicit FDs for maintenance relations

Relation	Determinant	Determined attributes
MAINTENANCE_RECORD	maintenance_record_id	space_id, reported_by_user_account_id, assigned_to_user_account_id, maintenance_status_id, impact_level_id, problem_description, start_time, completion_time, result_note
MAINTENANCE_IMPACT_EVENT	maintenance_impact_event_id	maintenance_record_id, old_impact_level_id, new_impact_level_id, changed_at

The current impact and impact-event history represent different facts. Their required agreement is a cross-row transaction rule, not a transitive FD inside either maintenance relation.

Association Relations

Table 15: Explicit FDs for association relations

Relation	Determinant	Determined attributes
SPACE_FACILITY	space_facility_id (space_id, facility_id)	space_id, facility_id space_facility_id
BOOKING_ADVISORY_ACKNOWLEDGEMENT	advisory_acknowledgement_id	booking_request_id, maintenance_record_id

Relation	Determinant	Determined attributes
	(booking_request_id, maintenance_record_ id)	advisory_acknowledgement_ id

Both association relations have a surrogate primary key and a composite alternate key. They store no copied descriptive attribute, so there is no partial or transitive dependency through either component of the composite key.

9.3 FD-Based 3NF Observation

Across all 18 relations, every non-trivial FD listed above has a candidate key as its determinant. There is therefore no documented pattern of the form $K \rightarrow B$ and $B \rightarrow C$, where both B and C are non-key attributes. This is the direct FD evidence used by Section 10 to conclude that every relation satisfies 3NF.

Rules such as booking non-overlap, out-of-service blocking, advisory acknowledgement, and current-impact/event synchronization involve multiple rows or relations. They are integrity rules enforced transactionally and are not functional dependencies within one relation.

10 Normalization Validation

The final Phase 2 schema is validated against First, Second, and Third Normal Form using the functional dependencies in Section 9. The proof considers only stored attributes and declared candidate keys; transactional business rules are handled separately.

10.1 Validation Criteria

Table 16: Normal-form criteria applied to the final schema

Normal form	Criterion	Evidence in the Phase 2 schema
1NF	Every attribute is atomic and no repeating group is stored in one row.	Facilities, impact events, and advisory acknowledgements are represented by separate relations rather than lists or repeating columns.
2NF	No non-prime attribute depends on only part of a composite candidate key.	Relations with single-attribute keys satisfy this automatically. The two association relations contain no attribute dependent on only one foreign-key component.
3NF	For every non-trivial FD $X \rightarrow A$, X is a superkey or A is prime.	Every determinant listed in Section 9 is a declared candidate key.

10.2 Relation-Level Validation

Instead of repeating three identical “Yes” columns for every relation, the relations are grouped below by the form of their proof. Every relation is still accounted for explicitly.

Lookup and configuration relations

Result: 3NF

ROLE, ACCOUNT_STATUS, SPACE_STATUS, BOOKING_STATUS, MAINTENANCE_STATUS, MAINTENANCE_IMPACT_LEVEL, and INSTANT_APPROVAL_SPACE_TYPE.

All values are atomic, and every determinant documented for these relations is a declared candidate key.

Core entity and fact relations

Result: 3NF

DEPARTMENT, USER_ACCOUNT, SPACE, FACILITY, BOOKING_REQUEST, APPROVAL_DECISION, USAGE_SESSION, MAINTENANCE_RECORD, and MAINTENANCE_IMPACT_EVENT.

Every non-key fact depends on a complete candidate key. No documented non-key attribute determines another non-key attribute.

Association relations

Result: 3NF

SPACE_FACILITY and BOOKING_ADVISORY_ACKNOWLEDGEMENT.

Each association is atomic. Its surrogate key and composite alternate key are candidate keys, and no stored fact depends on only one component of the composite key.

10.3 Important Design Checks

Association data. Facility details remain in FACILITY, and space details remain in SPACE. The junction relation stores only the association identifiers.

Advisory acknowledgement. One row represents one booking acknowledging one maintenance record. Requester and maintenance descriptions are not copied into the association relation.

Maintenance impact. MAINTENANCE_RECORD stores the current impact, while MAINTENANCE_IMPACT_EVENT stores historical changes. They are distinct current-state and event-history facts. Their agreement is a transactional integrity rule, not a transitive FD inside either relation.

Approval history. APPROVAL_DECISION has its own identifier. The booking foreign key is deliberately non-unique, allowing decision history without incorrectly treating a booking as the determinant of one decision row.

10.4 Final Result

All 18 relations in the final Phase 2 design satisfy at least Third Normal Form under the documented functional dependencies.

No Phase 2 decomposition is required. Earlier draft relations that were not supported by the requirement, including `APPROVAL_METHOD` and `ACADEMIC_SEMESTER`, are not part of the final schema and therefore are not included in this validation.

11 Agent and LLM Improvement Process

11.1 LLM Models

The group used `openai/gpt-5.5` throughout the Phase 2 workflow, including requirement-change analysis, database-design updates, schema migration, concurrency design and implementation, concurrency testing, data generation, analytical queries, index tuning, validation, and report preparation.

11.2 Improvements from Phase 1

The agents were improved through repeated requirement review, artifact comparison, SQL execution, and correction. Table [17](#) summarizes the main mistakes encountered and how the workflow was adjusted.

Table 17: Agent mistakes and resulting improvements

Problem encountered	Improvement applied
Early designs went beyond the requirement by adding semester and approval-method tables, permission hardening, retry/deadlock taxonomies, and large test matrices.	The agents were instructed to justify every addition from the requirement and keep the solution proportional to a student demonstration. Accepted demo decisions are carried forward instead of being reopened repeatedly.
Different artifacts used inconsistent definitions for time zones, active maintenance, occupancy, approval history, and automatic approval.	Shared vocabulary was added to the agent instructions: Vietnam-local time; Reported/In progress as active maintenance; approved/checked_in as current occupancy; approved decision history for reports; and a dedicated System actor.
Initial concurrency drafts separated availability checking from approval, mixed lock orders, or relied on the test session to acquire the lock.	All protected procedures now acquire their own transaction-owned, per-space application lock in the same space-first order. The safe test proves that A commits, B rechecks, detects the overlap, returns 52103, and leaves no committed conflict.
Generated data contained unrealistic life-cycles: completed and no-show rows lacked prior approval, rejected rows lacked a complete rejection decision, and active maintenance could contain completion facts.	The generator and its validation gate were expanded to verify approval history, usage sessions, rejection reasons, maintenance state, advisory acknowledgement, actor eligibility, and non-overlap before the dataset can be used.
Performance sections originally contained placeholders or measurements that were not sufficiently reproducible.	The tuning workflow now uses the same validated dataset, parameters, and result set before and after indexing, with a warm-up pass, STATISTICS IO/TIME , actual-plan evidence, and no unsupported performance claim.
Responsibility sometimes drifted between design, implementation, testing, and tuning stages.	Artifacts 08–16 now have explicit owner agents, declared upstream inputs, and stage-specific self-checks. The reviewer validates independently without silently rewriting another owner’s artifact.

The main lesson was that better results came from tighter traceability and executable validation, not from adding more complexity. Assumptions, proposed elements, and unresolved questions are now recorded explicitly so later agents do not silently reinterpret earlier decisions.